JAIST 西村研究室セミナー向け

ONTOLOGY × KNOWLEDGE GRAPH

オントロジーと知識グラフ

LLM・AI Agent・Spatial AI への展開
意味、関係、文脈を AI の判断と行動へつなぐ

AIエンジニアが仕事から得られた問題意識や取り組みがベースとなっております

2026.11.09 改訂｜20–25 min + Q&A

AIは「情報」だけでなく「意味」を必要としている

生成から判断・行動へ進むほど、組織固有の概念・関係・制約・現在状態が重要になる。

01
DATA
文書・DB・SaaS・センサーデータ

→

02
MEANING
概念・関係・指標・ルール

→

03
CONTEXT
いま重要なこと・許されること

→

04
ACTION
回答・判断支援・ツール実行

意味・関係・制約を明示した知識基盤

DIKW | 基礎

DIKWで見ると、意味・関係・Contextがデータを行動へ変える

データを解釈・構造化・選択することで、AI Agentが判断と実行に使える知識になる。

WISDOM / ACTION
Context Layer → AI Agent
目的・状況・権限に合う知識を選び、判断・計画・実行へ

KNOWLEDGE
Ontology ＋ Knowledge Graph
意味・制約を定義し、具体的なエンティティと事実を接続

INFORMATION
Semantic Layer
生データを共通の業務概念・指標として解釈

DATA
文書・DB・SaaS・センサー・業務イベント
分散した記録、数値、文章、状態変化

各レイヤーの役割

Semantic Layer
データを共通の業務用語・指標で読める形にする。

Ontology ＋ KG
意味と制約を定義し、具体的事実と関係を構造化する。

Context Layer ＋ Agent
必要な知識を構成し、回答・判断・ツール実行に使う。

下から上へ： データ → 解釈可能な情報 → 関係を持つ知識 → 目的に応じた判断・行動

※ DIKWとの対応は厳密な標準ではなく、本資料における機能上の整理。

オントロジーは「何が存在し、どう関係するか」を定義する

“概念化の明示的な仕様”

Gruber, 1993

暗黙知を、共有・再利用・機械処理できる概念体系へ。

用語集ではなく、概念階層・関係・制約まで形式化する。

CLASS
概念
設備、ポンプ、工程
対象の種類を定義

INSTANCE
個体
ポンプ P101、工程 A
現実の具体的対象

PROPERTY
属性・関係
所属、依存、状態
対象同士を接続

AXIOM
制約・公理
ポンプは設備の一種
推論・整合性確認

共有可能な意味モデルが、システム間の解釈差を減らす。

企業の競争領域は「データ」から「Context Layer」へ広がる

企業・技術	意味モデル／グラフの取り組み	Agent・業務への提供価値
Palantir Ontology	業務オブジェクト・関係・アクションを一体化	意味モデルと操作・権限を業務フローへ接続
AWS Context / COA	推定した関係と、専門家が承認するOWL/RDFのKG	MCP経由で検証可能な企業文脈をAgentに提供
Snowflake Cortex	Semantic Views＋KGノード/エッジ＋GraphRAG	オントロジー階層を探索しSQL・Cortex Agentを補強
Databricks Genie Ontology	Unity Catalogの意味定義＋自動推定コンテキスト	指標・定義・関係をGenie Agentに共通提供
NVIDIA Auto Ontology	メタデータから用語・SQL属性・関係を生成	人が認証した文脈をMCP / APIでAgentへ公開

共通点：データだけでなく、定義・関係・来歴・権限をContextとして提供。
注意：方式・成熟度は異なる。SnowflakeはエンジニアリングSnowflakeはエンジニアリング実装例、Genie OntologyはPublic Preview。

出典：AWS / Snowflake / Databricks / NVIDIA 各社公式（末尾にURL）

生成AIの普及が、意味と関係を管理する新たな需要を生んだ

01
データの分散

文書・DB・SaaS・IoTに知識が散在。

必要：意味と関係の統合

02
LLMの普及

文書からエンティティと関係を抽出可能に。

必要：知識構築の支援

03
Agentの実用化

検索・回答から、判断と業務実行へ。

必要：権限・履歴・最新状態

知識表現は不要になるのではない。AIが使える形へ再設計される。

TREND｜なぜ今、AIエンジニアが注目するのか

生成AIが「知識の意味・関係・信頼性」を再び重要にした

2026年はGraphRAG・Agent・企業Context Layer・自動オントロジー構築が接点になる。

01

検索から行動へ

単なる検索結果でなく、
実行可能な操作・制約・
承認条件が必要。

02

LLMで抽出を支援

文書から概念や関係を
候補生成できるが、
誤抽出・同一性が課題。

03

企業固有の意味

売上、顧客、設備などの
定義を複数システムで
統一して活用する。

04

物理空間へ拡張

3D Scene GraphとKGを
結び、空間認識から
計画・行動へつなぐ。

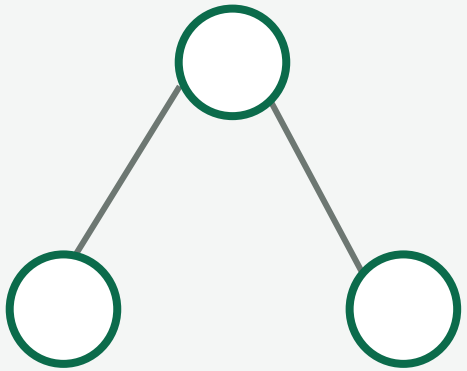
論点：KGを「作れるか」だけではなく「更新・検証・権限付きで使い続けられるか」。

出典：NVIDIA Technical Blog / Databricks Genie Ontology

グラフの価値は「関係」を探索・分析できることにある

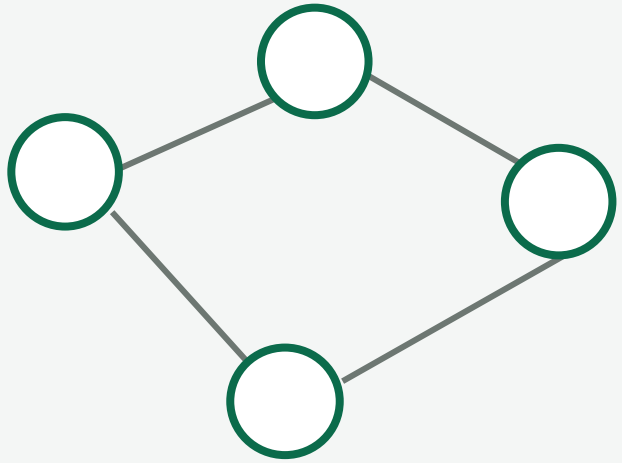
木構造

階層・索引・分岐に強い



一般グラフ

多対多・循環・複数種類の関係



応用できる分析

最短経路

依存・到達可能性

中心性

コミュニティ検出

交通、製造、ソフトウェア、知識表現——現実の複雑さを「関係の構造」として扱う。

グラフが関係を表し、オントロジーが意味を与える

ONTOLOGY

「ポンプ」は「設備」の一種 ／ 「設備」は「工程」に属する ／ 設備は別の設備に依存し得る

KNOWLEDGE GRAPH

P101 → 所属 → 工程A　　P101 → 依存 → 電源B　　P101 → 状態 → 停止中

CONTEXT LAYER

問い： P101停止時の影響工程・関連設備・適用ルールは何か？
目的に必要な知識だけを選び、AIへ渡す。

※ 4用語は厳密な標準階層ではなく、機能上の整理。

KNOWLEDGE MODELING | RDF / OWL / SHACL

グラフ・オントロジー・検証は、役割を分けて考える

「関係を保存する」「意味を定義する」「制約を検査する」は別々の機能。

RDF

主語–述語–目的語
IRIによる識別、SPARQLで検索

RDFS / OWL

クラス・プロパティ・公理
階層・論理的含意を表現

SHACL

値の型・必須項目・多重度
RDFデータの品質を検証

Property Graph

ノード・エッジの属性
Cypher / Gremlin等で探索

例：設備 P101 — depends_on → 電源 B
関係の事実はRDF／Property Graphへ。depends_on の意味はオントロジー、IDや状態値の整合性はSHACLで検査する。

重要：OWLのオープンワールド推論とSHACLのデータ検証を混同しない。

出典：W3C PROV-O / LinkML RDF documentation

LinkMLで概念・属性・関係をYAMLからモデル化する

データベースに依存しないスキーマを、RDFやアプリ実装に橋渡しする。

```
classes:
  Equipment:
    slots: [id, depends_on]
slots:
  id:
    identifier: true
    range: string
  depends_on:
    range: Equipment
    multivalued: true
```

→

JSON Schema / Pydantic

JSON-LD / RDF / OWL

SHACL → RDFバリデーション

注意：LinkMLが自動で因果効果を推定するわけではない。因果変数や根拠を表す共通スキーマとして利用できる。

出典：LinkML（https://linkml.io/）

NVIDIA | AUTO ONTOLOGY

NVIDIA Auto Ontologyは、企業メタデータから文脈を生成する

ユーザーがいう「Auto Graph」に近い技術として、公式名称はNVIDIA Auto Ontology。

01
DISCOVER
DBスキーマ・列・
クエリ履歴を解析

→

02
GENERATE
用語・指標SQL・
結合関係を候補化

→

03
CERTIFY
担当者が定義を
レビュー・認証

→

04
SERVE
MCP・CLI・APIで
Agentに意味を配信

技術例：PostgreSQL + pgvector、Text-to-SQL、接続元にSnowflake / Databricks等。
設計原則：自動生成された関係を、そのまま「真実」として実行しない。

※ ArangoDBの「AutoGraph Studio」とは別プロジェクト。

出典：NVIDIA Auto Ontology公式GitHub

GraphRAGは「近い文章」から「つながる知識」へ検索を拡張する

SOURCES
文書・DB・業務イベント・会話履歴

→

ONTOLOGY + KG
概念・関係・属性・履歴・出典

→

GRAPH RAG
Vector検索 + Traversal + Rerank + Context構成

→

LLM / AGENT
回答・推論・計画・ツール実行

観点	Vector RAG	GraphRAG
検索単位	意味的に近いチャンク	エンティティ・関係・コミュニティ
得意な質問	局所的事実・類似文章	複数文書横断・関係探索・全体テーマ
主な課題	関係の取りこぼし	構築・更新コスト、抽出誤り

目的はKGそのものではない。AIの目的と行動に合うContextを提供すること。

ENGINEERING | IMPLEMENTATION

実装は「抽出→検証→検索→実行」を分離する

オントロジーの更新を、推論・検索の品質と本番運用までつなぐ。

SOURCES	INGEST	MODEL	STORE	RETRIEVE	AGENT
DB・文書 IoT・イベント	Entity抽出 ID解決・更新	LinkML・OWL SHACLで検証	RDF / Property Graph + Vector	SPARQL / Cypher Vector + Rerank	Context構成 承認・ツール実行

横断的なガバナンス

ID / 同一性 | provenance・出典 | valid time・鮮度 | 権限・PII | 監視・再構築

スキーマ・ルールはGitで版管理し、生成・評価・承認のワークフローをCI/CDに組み込む。

出典：LinkML + W3C PROV-O + AWS Context Ontology Accelerator

ENGINEERING | EVALUATION

GraphRAGやOntologyは、答えの品質で評価する

可視化の美しさよりも、誤関連・誤回答・誤実行をどれだけ減らすかを測る。

評価層	主な指標	確認する失敗
グラフ構築	Entity Linking F1、関係 Precision / Recall	重複ノード、誤った因果ラベル
意味・整合性	SHACL違反数、スキーマ互換性	型違反、孤立ノード、定義の矛盾
検索・回答	Recall@k、Faithfulness、出典一致率	誤トラバーサル、未根拠回答
運用・安全	鮮度、ACL漏洩0、遅延・コスト	古い状態、権限外参照、誤実行

実験：Vector RAG ／ Hybrid RAG ／ GraphRAG を同じ質問集・正解・制約で比較。

出典：NVIDIA KG evaluation / LinkML validation

AI開発は単一ループから、知識と仕事を結ぶグラフ設計へ

※AI Coding Agentの世界では急速に変化があります

Loop Engineering → Graph Engineeringへ

Loop ENGINEERING

PLAN → CODE → TEST → REVIEW ↺

反復・停止条件・検証・エスカレーション

GRAPH ENGINEERING

条件分岐、並列処理、承認、再実行を依存関係と状態遷移で設計する。

Knowledge Graph

要件・仕様・ADR・コード・テストの関係

Workflow Graph

実装・レビュー・テスト・承認・リリース

Dependency Graph

モジュール・API・ライブラリ間の依存

※ Graph Engineeringは新興の実務的呼称。本資料では複数グラフを接続し、Agentの知識と実行を設計する視点として用いる。

09｜実世界への展開

知識グラフは複雑な社会課題を「関係の構造」に変える

製造業

課題：故障の影響範囲が不明

設備・工程・故障・手順を接続

医療・介護

課題：支援情報が分散

状態・支援者・サービスを権限付きで関連付け

地域創生

課題：地域資源とニーズが未接続

企業・研究者・技術・地域課題を接続

サプライチェーン

課題：供給停止の波及が不明

企業・部品・物流経路の依存を表現

製造業における故障対応

1｜設備停止を検知 → 2｜依存設備・影響工程を探索 → 3｜履歴・手順・安全制約を取得 → 4｜人間承認後に保守作業を起票

Spatial AIは、幾何情報に「空間的・意味的な関係」を重ねる

RGB画像	外観・色・テクスチャ 距離と3次元関係は弱い
Depth / Point Cloud	距離・形状・3次元座標 意味情報は弱い
Semantic Map	物体・領域の意味分類 関係や階層は限定的
3D Scene Graph	物体・場所・空間関係・意味関係を統合

指示理解の例

「キッチンの机の上にあるコップを取ってきて」

場所階層：建物 → キッチン
物体関係：コップ → on_top_of → 机
計画：移動 → 探索 → 把持

研究例： Hydraは階層的3D Scene Graphをリアルタイム構築。
ConceptGraphsは2D基盤モデルの出力を3Dへ統合する。

RESEARCH | CAUSAL KG

「つながっている」と「原因である」を区別する

KGの関係から、介入による変化を直接導けるわけではない。

Knowledge Graph（事実・意味）

ポンプP101 → depends_on → 電源B
P101 → belongs_to → 工程A
関係は、依存・所属・階層・履歴も含む。

Causal Graph（原因・介入）

電源喪失 → モータ停止 → 流量低下
交絡・時間順序・仮定を明示する。
因果効果は統計モデル／実験で推定する。

相関の問い $P(Y \mid X)$　≠　介入の問い $P(Y \mid do(X))$

RDF / OWLは因果モデル・仮定・出典の表現に使えるが、do-calculusや因果効果推定の代替ではない。

出典：SoCK (Semantic Web, 2026) / Causal Knowledge Graph research

RESEARCH | PROTOTYPE

Ontology × RDF × 因果推論の接続を研究テーマにする

設備停止・工程影響の例で、小さく反実仮想・介入推定まで評価する。

STEP 1　意味を定義

LinkMLでEquipment / Event / State / Timeを設計。RDF化。

STEP 2　因果候補を作る

専門家の仮説 + 時系列 + causal discoveryを突合。

STEP 3　因果モデル化

DAG / SCMに交絡・時間・対象集団・介入を定義。

STEP 4　効果を検証

DoWhy等で識別・推定・反証。介入ログで妥当性確認。

比較条件：KGだけ／因果グラフ併用。評価：影響範囲特定、介入効果、誤因果率、説明可能性。

既存研究：SoCKは複数の因果グラフをRDFとオントロジーで統合し、仮定・出典・構造差を扱う。

出典：SoCK (2026), SAGE / doi:10.3233/SSW260004

RESEARCH AGENDA | まとめ

研究の焦点：更新・Context構成・実世界と因果

01
動的な更新
LLMで新知識を取得し、矛盾・履歴・出典を管理しながらOntology / KGを更新できるか。

02
Context構成
目的・時刻・状況・権限に合う情報を選び、信頼性・鮮度・関連性をどう評価するか。

03
実世界・因果接続
KG・3D Scene Graph・因果モデルを結び、ロボットの理解・計画・介入を支援できるか。

知識を表現する研究から、AIが利用し、判断し、実世界で行動するための研究へ。

Knowledge Science × AI × Robotics

REFERENCES｜公式資料・先行研究

参照した資料と、次に読むべき情報源

以下は2026年11月9日時点の確認情報。プレビューや実験的機能は変更され得る。

- NVIDIA Auto Ontology
- Snowflake｜Ontology-grounded Cortex Agents
- NVIDIA｜Agent Knowledge Foundation
- Databricks｜Genie Ontology (Public Preview)
- LinkML｜公式ドキュメント
- W3C｜PROV-O / 出典・来歴
- LinkML｜RDF / SHACL / OWL
- SoCK｜Synthesising Causal Knowledge (2026)
- AWS｜Context Ontology Accelerator
- Causal KG｜adverse drug effects (2025)

※ NVIDIA Auto OntologyとArango AutoGraph Studioは別物。SnowflakeのKG例は公式エンジニアリング記事の実装例。